Автономная некоммерческая образовательная организация высшего образования

«Научно-технологический университет «Сириус»»

Лабораторная работа № 2

«Регистрация показаний аналоговых датчиков»

Вариант 1

По дисциплине: Основы программирования для микроэлектроники

Выполнил:

Студент

Арунова Маргарита Дмитриевна

Проверил:

_____

ФТ Сириус, 2026 г.

Содержание

Тема

Цель работы

Оборудование

Схема подключения

Описание алгоритма

Основные формулы

- Преобразование к температуре
- Модифицированное скользящее среднее

Результат работы программы

Вывод

Тема

Регистрация показаний аналогового датчика температуры с последующим сглаживанием результатов.

Цель работы

Изучить методы считывания аналоговых сигналов с датчика температуры LM35, организовать сбор выборки фиксированного объема по внешнему запуску и выполнить сглаживание результатов методом модифицированного скользящего среднего.

Оборудование

- Arduino Uno
- датчик температуры LM35
- кнопка запуска
- макетная плата
- соединительные провода

Схема подключения

- LM35 VCC -> 5V
- LM35 VOUT -> A0
- LM35 GND -> GND
- Кнопка -> D2 и GND

Описание алгоритма

После подачи питания микроконтроллер находится в режиме ожидания внешнего сигнала запуска на входе D2. По прерыванию начинается процесс выборки. Сигнал датчика температуры считывается через A0 с частотой 10 Гц, а всего сохраняется 64 отсчета. Каждое значение преобразуется в температуру в градусах Цельсия. Для сглаживания используется модифицированное скользящее среднее, которое рассчитывается рекурсивно по предыдущему сглаженному значению.

Основные формулы

Преобразование к температуре

$$U = ADC * 5 / 1023$$

$$T = U * 100$$

Для LM35 коэффициент преобразования составляет 10 мВ / °C.

Модифицированное скользящее среднее

$$Y(k) = Y(k-1) + (X(k) - Y(k-1)) / N$$

где:

- X(k) - текущее измерение;
- Y(k) - сглаженное значение;
- N - параметр окна, в работе принят равным 5.

Результат работы программы

Программа формирует три массива:

- исходные значения АЦП;
- значения температуры без сглаживания;

- сглаженные значения температуры.

После завершения сбора они остаются в памяти микроконтроллера и могут быть считаны средствами отладки. При поступлении нового внешнего сигнала программа выполняет повторную выборку с обновлением содержимого массивов.

Вывод

В ходе лабораторной работы была реализована система регистрации температуры с внешним запуском. Полученные данные сохраняются в памяти микроконтроллера, а применение модифицированного скользящего среднего уменьшает влияние случайных колебаний измеряемого сигнала.

Таблица соединений

Элемент	Подключение
LM35	VCC -> 5V, VOUT -> A0, GND -> GND
Кнопка запуска	D2 -> кнопка -> GND
Отладка	Просмотр массивов через Serial/Debugger

Иллюстрации и скриншоты

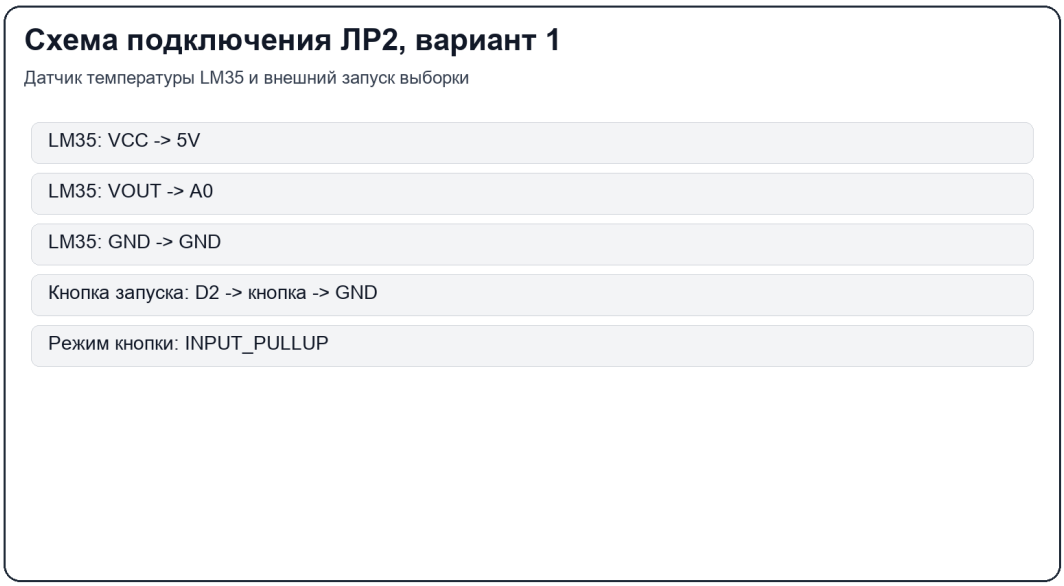

Рисунок 1 - Схема подключения LM35

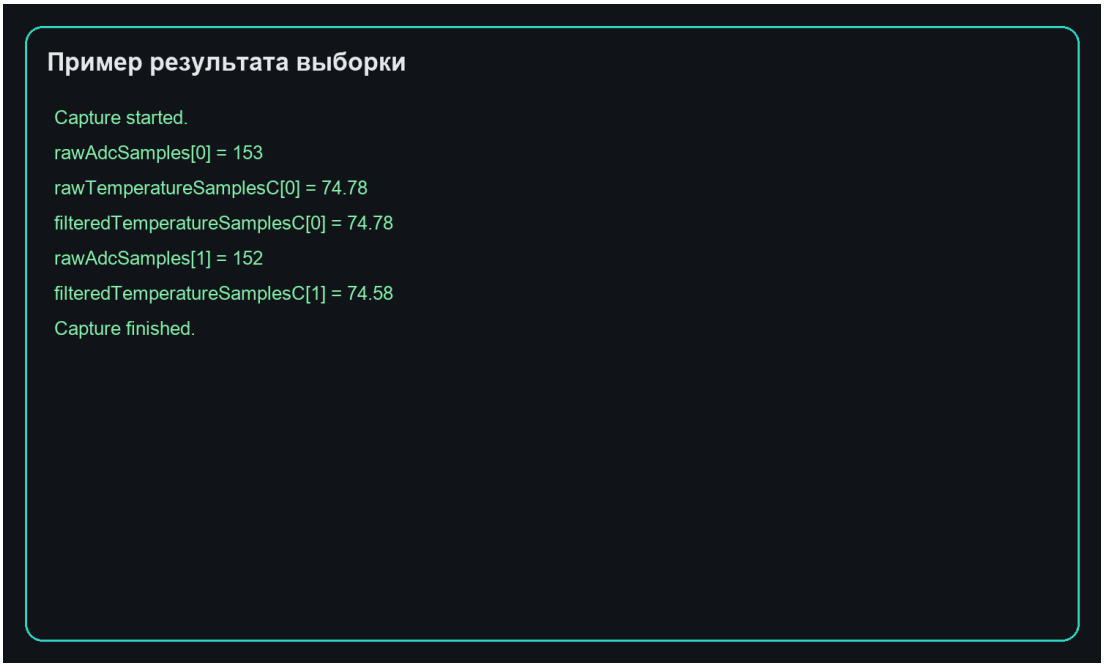

Рисунок 2 - Пример результатов выборки

Приложение. Листинг программы

lab2_variant1.ino

```
#include <Arduino.h>
```

```
const uint8_t LM35_PIN = A0;
const uint8_t TRIGGER_PIN = 2;
const uint8_t STATUS_LED_PIN = LED_BUILTIN;

const uint16_t SAMPLE_FREQUENCY_HZ = 10;
const uint8_t SAMPLE_COUNT = 64;
const uint8_t MMA_WINDOW = 5;
const unsigned long SAMPLE_INTERVAL_MS = 1000UL / SAMPLE_FREQUENCY_HZ;

volatile bool startSignalReceived = false;

uint16_t rawAdcSamples[SAMPLE_COUNT];
float rawTemperatureSamplesC[SAMPLE_COUNT];
float filteredTemperatureSamplesC[SAMPLE_COUNT];

bool acquisitionInProgress = false;
bool acquisitionFinished = false;
uint8_t currentSampleIndex = 0;
unsigned long lastSampleTimestamp = 0;

void onStartSignal() {
  startSignalReceived = true;
}

float adcToTemperatureC(uint16_t adcValue) {
  float voltage = (adcValue * 5.0f) / 1023.0f;
  return voltage * 100.0f;
}

float applyModifiedMovingAverage(float newValue, float previousFilteredValue, bool
isFirstSample) {
  if (isFirstSample) {
    return newValue;
  }

  return previousFilteredValue + (newValue - previousFilteredValue) / MMA_WINDOW;
}
```

```
void resetCaptureState() {
  acquisitionInProgress = true;
  acquisitionFinished = false;
  currentSampleIndex = 0;
  lastSampleTimestamp = millis();
  digitalWrite(STATUS_LED_PIN, HIGH);
}

void finalizeCapture() {
  acquisitionInProgress = false;
  acquisitionFinished = true;
  digitalWrite(STATUS_LED_PIN, LOW);
}

void sampleOnce() {
  uint16_t rawAdc = analogRead(LM35_PIN);
  float rawTemperature = adcToTemperatureC(rawAdc);
  float previousFiltered = currentSampleIndex == 0 ? rawTemperature :
filteredTemperatureSamplesC[currentSampleIndex - 1];
  float filteredTemperature = applyModifiedMovingAverage(rawTemperature,
previousFiltered, currentSampleIndex == 0);

  rawAdcSamples[currentSampleIndex] = rawAdc;
  rawTemperatureSamplesC[currentSampleIndex] = rawTemperature;
  filteredTemperatureSamplesC[currentSampleIndex] = filteredTemperature;

  currentSampleIndex++;

  if (currentSampleIndex >= SAMPLE_COUNT) {
    finalizeCapture();
  }
}

void setup() {
  pinMode(TRIGGER_PIN, INPUT_PULLUP);
  pinMode(STATUS_LED_PIN, OUTPUT);
  digitalWrite(STATUS_LED_PIN, LOW);

  attachInterrupt(digitalPinToInterrupt(TRIGGER_PIN), onStartSignal, FALLING);
```

```
  Serial.begin(9600);
  Serial.println(F("Lab 2 / Variant 1 ready."));
  Serial.println(F("Waiting for external start signal on D2."));
}


void loop() {
  if (startSignalReceived && !acquisitionInProgress) {
    noInterrupts();
    startSignalReceived = false;
    interrupts();


    resetCaptureState();
    Serial.println(F("Capture started."));
  }


  if (acquisitionInProgress) {
    unsigned long now = millis();


    if (now - lastSampleTimestamp >= SAMPLE_INTERVAL_MS) {
      lastSampleTimestamp += SAMPLE_INTERVAL_MS;
      sampleOnce();


      if (acquisitionFinished) {
        Serial.println(F("Capture finished. Inspect arrays in debugger:"));
        Serial.println(F("rawAdcSamples, rawTemperatureSamplesC,
filteredTemperatureSamplesC"));
      }
    }
  }
}
```